

Faculty of Computing
CS110: Fundamentals of Computer Programming
Class: BSAI 2026A

Lab 04: Operator Precedence, Financial Application, and Bitwise Operators

CLO 1

Understand the syntax and semantics of different programming constructs

CLO 3

Build a program and associated documentation using appropriate IDE and supplementary tools

Time: 2:00 pm - 4:50 pm

Instructor: Sahar Arshad
Lab Engineer: Urooj Akmal



Introduction:

This lab continues the study of C++ operators through three focused applications. Students will first examine how operator precedence controls the evaluation of a mathematical expression, then use the sizeof operator and arithmetic calculations to estimate memory requirements for a financial application, and finally apply bitwise operators to two 8-bit values. The lab is intentionally limited to three assessed tasks so that each program can be compiled, tested, explained, and documented carefully.

Learning Objectives:

After completing this lab, you will be able to:

- Apply operator precedence and parentheses correctly when translating a mathematical expression into C++.
- Use suitable numeric data types and the sizeof operator to estimate memory usage for a small financial record and for multiple records.
- Apply the bitwise operators &, |, ^, <<, and >> and display 8-bit binary results using std::bitset.
- Build, run, test, and document C++ programs in the IDE using clear variable names, comments, and output screenshots.

Tools/Software Requirement:

- The C++ IDE installed and configured in earlier labs (for example, Microsoft Visual Studio or the IDE specified by your instructor).
- A working C++ compiler integrated with the IDE.
- MS Word or equivalent for preparing the lab report.
- The standard C++ <bitset> library for displaying binary values in Task 3.

Important: All three tasks in this lab are compulsory and must be submitted.

Task 1 [CLO 1] - Operator Precedence and Expression Evaluation (45 minutes):

Write a C++ program that evaluates the following mathematical expression. The purpose is to translate the expression carefully and observe how multiplication, division, addition, subtraction, and parentheses determine the order of evaluation.

$$\text{result} = 4 / [3(r + 34)] - 9(a + bc) + [3 + d(2 + a)] / (a + bd)$$

1. Declare a, b, c, d, r, and result using a numeric data type that preserves decimal values. Briefly justify your choice in a comment.
2. Read the five values from the user. Use the required test values: a = 5, b = 3, c = 4.5, d = 6, and r = 6.12.
3. Translate the mathematical expression into a valid C++ expression. Use * explicitly for multiplication and use parentheses to make the intended grouping clear.
4. Before running the program, write the expected evaluation order in your report. Identify which bracketed/grouped expressions are evaluated first, where multiplication occurs before addition, and when the three major terms are combined.
5. Print the final value with a clear label. Then rewrite the same expression with extra parentheses that make the evaluation order visually explicit, calculate it again, and verify that both versions produce the same result.
6. In 3-4 sentences, summarize how precedence and parentheses affected the evaluation. Also state why a floating-point type is appropriate for this expression.

Suggested C++ expression format:

```
result = 4.0 / (3.0 * (r + 34.0))  
        - 9.0 * (a + b * c)  
        + (3.0 + d * (2.0 + a)) / (a + b * d);
```

Deliverable: C++ code, the written evaluation order, one screenshot of the required test run, and the short precedence/data-type explanation.



Task 2 [CLO 1, CLO 3] - Financial Application: Memory Usage Analyzer (45 minutes):

Develop and execute a C++ program that estimates the memory used to store basic account information in a financial application. This task combines the sizeof operator with arithmetic calculations and unit conversion.

1. Declare one sample variable of each type: int, float, double, char, and bool. Assign a sensible example value to each variable.
2. Use sizeof() to display the memory size in bytes of each of the five variables. Label every line of output clearly.
3. Assume one account record contains these four fields: int accountNumber, float transactionAmount, char accountStatus, and bool isActive.
4. Estimate the memory for one account by adding the sizeof() value of the four fields. Display the result in bytes.
5. Estimate the memory required for 10,000 accounts. Display this total in bytes, kilobytes, and megabytes using 1 KB = 1024 bytes and 1 MB = 1024 KB. Use floating-point division for the KB and MB values.
6. Add at least two useful comments and use meaningful identifiers such as bytesPerAccount, totalBytes, totalKB, and totalMB.
7. In your report, explain in 2-3 sentences what sizeof() tells you and why the total for 10,000 accounts is much larger than the total for one account.

Scope note: For this introductory task, estimate one account record by adding the sizes of its individual fields. Actual structure/class memory may include padding or alignment, which is beyond the scope of this lab.

Deliverable: C++ code, one output screenshot showing the individual data-type sizes and the account-memory calculations, and the short explanation of sizeof() and the 10,000-account estimate.

Task 3 [CLO 1, CLO 3] - Bitwise Data Compression Simulator (55 minutes):

Construct and implement a C++ program that works with two unsigned 8-bit values. The program will display each input in binary and then apply common bitwise operators. For this lab, enter values only in the range 0-255.

1. Include both <iostream> and <bitset>. Declare two variables as unsigned int and read values a and b from the user.
2. Display a and b in decimal and as 8-bit binary values using std::bitset<8>.
3. Calculate and display the following operations in both decimal and 8-bit binary form: a & b, a | b, a ^ b, a << 2, and b >> 3.
4. For every operation, add a clear label so the output identifies the operator being demonstrated.
5. Required Test 1: a = 25 and b = 12. Required Test 2: a = 60 and b = 32. Predict the binary result of at least the AND, OR, and XOR operations before running each test.
6. In your report,



```
#include <iostream>
#include <bitset>

int main() {
    unsigned int a, b;

    std::cout << "Enter first 8-bit integer (0-255): ";
    std::cin >> a;
    std::cout << "Enter second 8-bit integer (0-255): ";
    std::cin >> b;

    std::cout << "A = " << a << " Binary: " << std::bitset<8>(a) << "\n";
    std::cout << "B = " << b << " Binary: " << std::bitset<8>(b) << "\n";

    // Complete the five required bitwise operations here.
    return 0;
}
```

Reminder: An unsigned 8-bit value ranges from 0 to 255. The required test values are chosen so that the displayed left-shift result remains easy to interpret within eight bits.

Deliverable: Completed C++ code, prediction work for both required tests, screenshots of both executions, and the five one-sentence operator explanations.

Deliverables:

Compile a single Word document and submit it on LMS. Organize the report under Task 1, Task 2, and Task 3. All three tasks are compulsory. For each task, include the requested C++ code, output/IDE screenshot(s), calculations or predictions where required, and the short explanation/observation. The submitted code must be the code demonstrated during the lab.



Lab Rubrics

Your Lab 4 will be graded according to the following rubrics.

Lab Rubrics for Lab 4					
Sr. No.	Assessment	Unacceptable (0 Marks)	Does Not Meet Expectations (1/2 Marks)	Meets Expectations (3/4 Marks)	Exceeds Expectations (5 Marks)
1	Operator semantics and expression evaluation (CLO1, PLO1)	The student did not submit the required work, submitted work that cannot be evaluated, or used unfair means/plagiarized work.	The student is unable to translate the mathematical expression correctly, apply precedence/parentheses, calculate memory values, or use the required bitwise operators without substantial help.	The student demonstrates partial understanding of precedence, sizeof-based calculations, and bitwise operations. Most work is attempted and programs can run with limited guidance, although some expression or output errors remain.	The student correctly applies precedence and parentheses, uses suitable data types and sizeof calculations, performs the required bitwise operations, interprets binary results, and explains program behavior accurately.
2	IDE usage, testing, program construction, and documentation (CLO3, PLO5)	The student did not submit the required work, submitted work that cannot be evaluated, or used unfair means/plagiarized work.	The student has difficulty creating, building, and running the programs in the IDE and provides incomplete test evidence, comments, screenshots, or documentation.	The student can build and run the required programs and complete the specified tests with limited help. Code, comments, screenshots, predictions/calculations, and explanations are mostly complete and organized.	The student independently builds, runs, tests, and documents all three programs; uses clear identifiers and comments; provides complete screenshots and required working; and presents a well-structured report.

Task 1

I believe the expression will be evaluated as follows:

```
4/(3 * (r+34)) term 1
- 9 * (a+b*c) term 2
+ ( (3+ d*(2+a))/(a+b*d)) term 3
```

inner most brackets will be evaluated for all 3 terms following the rules of PEMDAS/BODMAS (Parenthesis, Exponents, multiplication, division, addition, subtraction). This will result in some cases where multiplication happens before division for example in term 2,

'b*c' will be evaluated before being added to 'a' following the conventions of PEMDAS

After brackets are evaluated the the following will happen:

term 1 - term 2 + term 3

which will evaluate to the final value to be stored in "result"



In this way, precedence is dictated by the rules of PEMDAS and more specifically, by using parentheses to group each term which results in an accurate translation of the given expression. Data type explanation attached in screenshot:

```
task1.cpp U X
nust > pf > LabTask04 > task1.cpp > main()
3  using namespace std;
4
5  int main () {
6      double a,b,c,d,r,result;
7      // the calculation can include decimal answers so to preserve the most amount of info,
8      // I'm using the 'double' data type as opposed to int
9      cout << "enter a value for a: " ;
10     cin >> a;
11     cout << "enter a value for b: " ;
12     cin >> b;
13     cout << "enter a value for c: " ;
14     cin >> c;
15     cout << "enter a value for d: ";
16     cin >> d;
17     cout << "enter a value for r: " ;
18     cin >> r;
19
20     result = 4/(3 * (r+34)) -9 * (a+b*c) + ( (3+ d*(2+a))/(a+b*d));
21     // 4/(3 * (r+34)) term 1
22     // -9 * (a+b*c) term 2
23     // + ( (3+ d*(2+a))/(a+b*d)) term 3
24     cout << "Result = " << result << endl;
25     return 0;
26 }
```

Problems Output Debug Console Terminal Ports Spell Checker 2

```
● tariqmahmood@Tariqs-MacBook-Air LabTask04 % ./task1
enter a value for a: 5
enter a value for b: 3
enter a value for c: 4.5
enter a value for d: 6
enter a value for r: 6.12
Result = -164.51
❖ tariqmahmood@Tariqs-MacBook-Air LabTask04 %
```



Task 2

Sample Variables size of:

```
task1.cpp U task2.cpp U X
nust > pf > LabTask04 > C- task2.cpp > main()
1 // LabTask04 -> task2.cpp
2
3 using namespace std;
4
5 int main () {
6     // using these variables as sample names as well to avoid redundancy
7     int accountNumber;
8     float transactionAmount;
9     double sampleDouble;
10    char accountStatus;
11    bool isActive;
12
13    accountNumber = 23102007;
14    transactionAmount = 6381.4;
15    accountStatus = 'A'; // A for active, U for unactive
16    isActive = true;
17
18    cout << "Variable " << "accountNumber" << " uses " << sizeof(accountNumber) << " bytes of memory" << endl;
19    cout << "Variable " << "transactionAmount" << " uses " << sizeof(transactionAmount) << " bytes of memory" << endl;
20    // commenting this out since its not needed anymore:
21    // cout << "Variable " << "sampleDouble" << " uses " << sizeof(sampleDouble) << " bytes of memory" << endl;
22    cout << "Variable " << "accountStatus" << " uses " << sizeof(accountStatus) << " bytes of memory" << endl;
23    cout << "Variable " << "isActive" << " uses " << sizeof(isActive) << " bytes of memory" << endl;
24
25    cout << "Total Memory usage fpor One account = " << sizeof(accountNumber) + sizeof(transactionAmount)
26    + sizeof(accountStatus) + sizeof(isActive) << endl;
27
28
29
30    return 0;
}

Problems Output Debug Console Terminal Ports Spell Checker
tariqmahmood@Tariqs-MacBook-Air LabTask04 % clang++ task2.cpp -o task2
tariqmahmood@Tariqs-MacBook-Air LabTask04 % ./task2
Variable accountNumber uses 4 bytes of memory
Variable transactionAmount uses 4 bytes of memory
Variable accountStatus uses 1 bytes of memory
Variable isActive uses 1 bytes of memory
Total Memory usage fpor One account = 10
tariqmahmood@Tariqs-MacBook-Air LabTask04 %
```



One Account SizeOf:

```
task1.cpp U task2.cpp U X
nust > pf > LabTask04 > G task2.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main () {
6      // using these variables as sample names as well tp avoid redundancy
7      int accountNumber;
8      float transactionAmount;
9      double sampleDouble;
10     char accountStatus;
11     bool isActive;
12
13     accountNumber = 23102007;
14     transactionAmount = 6381.4;
15     accountStatus = 'A'; // A for acctive, U for unactive
16     isActive = true;
17
18     cout << "Variable " << "accountNumber" << " uses " << sizeof(accountNumber) << " bytes of memory" << endl;
19     cout << "Variable " << "transactionAmount" << " uses " << sizeof(transactionAmount) << " bytes of memory" << endl;
20     cout << "Variable " << "sampleDouble" << " uses " << sizeof(sampleDouble) << " bytes of memory" << endl;
21     cout << "Variable " << "accountStatus" << " uses " << sizeof(accountStatus) << " bytes of memory" << endl;
22     cout << "Variable " << "isActive" << " uses " << sizeof(isActive) << " bytes of memory" << endl;
23
24
25
26     return 0;
27 }
```

Problems Output Debug Console Terminal Ports Spell Checker 3

```
tariqmahmood@Tariqs-MacBook-Air LabTask04 % clang++ task2.cpp -o task2
tariqmahmood@Tariqs-MacBook-Air LabTask04 % ./task2
Variable accountNumber uses 4 bytes of memory
Variable transactionAmount uses 4 bytes of memory
Variable sampleDouble uses 8 bytes of memory
Variable accountStatus uses 1 bytes of memory
Variable isActive uses 1 bytes of memory
tariqmahmood@Tariqs-MacBook-Air LabTask04 %
```



10,000 Accounts SizeOf:

```
task1.cpp U task2.cpp U X
nust > pf > LabTask04 > G- task2.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main () {
6      // using these variables as sample names as well tp avoid redundancy
7      int accountNumber;
8      float transactionAmount;
9      double sampleDouble;
10     char accountStatus;
11     bool isActive;
12
13     accountNumber = 23102007;
14     transactionAmount = 6381.4;
15     accountStatus = 'A'; // A for acctive, U for unactive
16     isActive = true;
17
18     cout << "Variable " << "accountNumber" << " uses " << sizeof(accountNumber) << " bytes of memory" << endl;
19     cout << "Variable " << "transactionAmount" << " uses " << sizeof(transactionAmount) << " bytes of memory" << endl;
20     // commenting this out since its not needed anymore:
21     // cout << "Variable " << "sampleDouble" << " uses " << sizeof(sampleDouble) << " bytes of memory" << endl;
22     cout << "Variable " << "accountStatus" << " uses " << sizeof(accountStatus) << " bytes of memory" << endl;
23     cout << "Variable " << "isActive" << " uses " << sizeof(isActive) << " bytes of memory" << endl;
24
25     int totalBytes = sizeof(accountNumber) + sizeof(transactionAmount)
26                     + sizeof(accountStatus) + sizeof(isActive); //take total here
27
28     double totalKiloBytes = totalBytes / 1024.0; //using floating poont division here by making numerator a float explicitly
29     double totalMegaBytes = totalKiloBytes / 1024.0;
30
31
32     cout << "Total Memory usage fpor One account in bytes = " << totalBytes << " B" << endl;
33     // multiply by 10k when outputting instead of in a variable
34     cout << "Total Memory usage for 10,000 accounts in bytes = "<< totalBytes * 10000 << " B" << endl;
35     cout << "Total Memory usage for 10,000 accounts in kilobytes = "<< totalKiloBytes * 10000 << " KB" << endl;
36     cout << "Total Memory usage for 10,000 accounts in Megabytes = "<< totalMegaBytes * 10000 << " MB" << endl;
37
38
39
40
41     return 0;
42 }
```

```
Problems Output Debug Console Terminal Ports Spell Checker 5
• tariqmahmood@Tariqs-MacBook-Air LabTask04 % clang++ task2.cpp -o task2
• tariqmahmood@Tariqs-MacBook-Air LabTask04 % ./task2
Variable accountNumber uses 4 bytes of memory
Variable transactionAmount uses 4 bytes of memory
Variable accountStatus uses 1 bytes of memory
Variable isActive uses 1 bytes of memory
Total Memory usage fpor One account in bytes = 10 B
Total Memory usage for 10,000 accounts in bytes = 100000 B
Total Memory usage for 10,000 accounts in kilobytes = 97.6562 KB
Total Memory usage for 10,000 accounts in Megabytes = 0.0953674 MB
• tariqmahmood@Tariqs-MacBook-Air LabTask04 %
```

“sizeof()” tells us the total amount of storage space in bytes for any given input. The total for 10,000 accounts is much larger than that for 1 account since it stores 10,000 times more information in memory (in terms of storage space).



Task 3

Test 1:

Prediction:

a = 25 in denary, 00011001 in binary
b = 12 in denary, 00001100 in binary

a & b = 00001000
a | b = 00011101
a ^ b = 00010101

Actual:

```
task1.cpp U task2.cpp U task3.cpp U X
nust > pf > LabTask04 > task3.cpp > main()
16 if (a > 255 || b > 255) {
17     cout << "Please enter values from 0 to 255." << endl;
18     return 1;
19 }
20
21 cout << "a as a decimal: " << a << endl;
22 cout << "b as a decimal: " << b << endl;
23 bitset<8> binary_a(a);
24
25 cout << "a as a Binary Number: " << binary_a << endl;
26
27 bitset<8> binary_b(b);
28 cout << "b as a Binary Number: " << binary_b << endl;
29
30 int operation1 = a & b;
31 int operation2 = a | b;
32 int operation3 = a ^ b;
33 int operation4 = a << 2;
34 int operation5 = b >> 3;
35
36 cout << "a & b in decimal: " << operation1 << endl;
37
38 cout << "a & b in binary: " << bitset<8>(operation1) << endl;
39 cout << "a | b in decimal: " << operation2 << endl;
40 cout << "a | b in binary: " << bitset<8>(operation2) << endl;
41
42 cout << "a ^ b in decimal: " << operation3 << endl;
43
44 cout << "a ^ b in binary: " << bitset<8>(operation3) << endl;
45
46 cout << "a << 2 in decima: " << operation4 << endl;
47 cout << "a << 2 in binry: " << bitset<8>(operation4) << endl;
48
49 cout << "b >> 3 in decimal: " << operation5 << endl;
50
51 cout << "b >> 3 in binary: " << bitset<8>(operation5) << endl;
52
53 return 0;
54 }

Problems Output Debug Console Terminal Ports Spell Checker
tarigmahmood@Tariqs-MacBook-Air LabTask04 % clang++ task3.cpp -o task3
tarigmahmood@Tariqs-MacBook-Air LabTask04 % ./task3
Enter a valur for a: 25
Enter a valur for b: 12
a as a decimal: 25
b as a decimal: 12
a as a Binary Number: 00011001
b as a Binary Number: 00001100
a & b in decimal: 8
a & b in binary: 00001000
a | b in decimal: 29
a | b in binary: 00011101
a ^ b in decimal: 21
a ^ b in binary: 00010101
a << 2 in decima: 100
a << 2 in binry: 01100100
b >> 3 in decimal: 1
b >> 3 in binary: 00000001
tarigmahmood@Tariqs-MacBook-Air LabTask04 %
```



Test 2:

a = 60 in denary, 00111100 in binary
b = 32 in denary, 00100000 in binary

a & b = 00100000
a | b = 00111100
a ^ b = 00011100

Actual:



```
task1.cpp U task2.cpp U task3.cpp U X
nust > pf > LabTask04 > task3.cpp > main()
16     if (a > 255 || b > 255) {
17         cout << "Please enter values from 0 to 255." << endl;
18         return 1;
19     }
20
21     cout << "a as a decimal: " << a << endl;
22     cout << "b as a decimal: " << b << endl;
23     bitset<8> binary_a(a);
24
25     cout << "a as a Binary Number: " << binary_a << endl;
26
27     bitset<8> binary_b(b);
28     cout << "b as a Binary Number: " << binary_b << endl;
29
30     int operation1 = a & b;
31     int operation2 = a | b;
32     int operation3 = a ^ b;
33     int operation4 = a << 2;
34     int operation5 = b >> 3;
35
36     cout << "a & b in decimal: " << operation1 << endl;
37
38     cout << "a & b in binary: " << bitset<8>(operation1) << endl;
39     cout << "a | b in decimal: " << operation2 << endl;
40     cout << "a | b in binary: " << bitset<8>(operation2) << endl;
41
42     cout << "a ^ b in decimal: " << operation3 << endl;
43
44     cout << "a ^ b in binary: " << bitset<8>(operation3) << endl;
45
46     cout << "a << 2 in decima: " << operation4 << endl;
47     cout << "a << 2 in binry: " << bitset<8>(operation4) << endl;
48
49     cout << "b >> 3 in decimal: " << operation5 << endl;
50
51     cout << "b >> 3 in binary: " << bitset<8>(operation5) << endl;
52
53     return 0;
54 }
```

Problems Output Debug Console Terminal Ports Spell Checker 9

```
• tariqmahmood@Tariqs-MacBook-Air LabTask04 % ./task3
Enter a valor for a: 60
Enter a valor for b: 32
a as a decimal: 60
b as a decimal: 32
a as a Binary Number: 00111100
b as a Binary Number: 00100000
a & b in decimal: 32
a & b in binary: 00100000
a | b in decimal: 60
a | b in binary: 00111100
a ^ b in decimal: 28
a ^ b in binary: 00011100
a << 2 in decima: 240
a << 2 in binry: 11110000
b >> 3 in decimal: 4
b >> 3 in binary: 0000100
❖ tariqmahmood@Tariqs-MacBook-Air LabTask04 %
```



& (bitwise AND) : returns 1 when both bits are 1.
| (bitwise OR) : returns 1 when at least one of the bits is 1.
^ (bitwise XOR): returns 1 when the two bits are unique.
<< (bitwise left shift): shifts specified amount of bits to the left and adds zeros on the right
>> (bitwise right shift) : shifts specified amount of bits to the right and removes the bits from the right.

Name: Abdur Rehman Tariq

CMS ID: 623486

Github Repo for all code/files:

<https://github.com/ARTariqDev/Nust-AI-2k26/tree/main/pf/LabTask04>